

Aula 4 — VPS, Docker, Kamal e deploy

Duração: ~3h · **Você sai daqui com:** a sua API no ar, em `https://seunome.capacita.<domínio>`, com deploy automático a cada push na `main`. **Checkpoint:** `git checkout aula-04`

Este roteiro é a versão para a turma do guia de infraestrutura do `seem-backend`. As decisões são as mesmas; a máquina é menor.

1. O que é uma VPS

VPS = *Virtual Private Server*. Um computador virtual, dentro de um servidor físico de um provedor, que é seu: você tem acesso root, escolhe o sistema, instala o que quiser.

Opção	O que você controla	O que você opera
Hospedagem compartilhada	quase nada	nada
PaaS (Heroku, Render, Fly)	o app	nada — mas paga mais e obedece as regras deles
VPS	tudo	tudo: SO, atualizações, firewall, banco
Kubernetes	tudo, em escala	muito mais

Por que VM com containers, e não Kubernetes

O `seem-backend` atende ~500 usuários num evento de uma semana. Kubernetes adicionaria custo e operação sem resolver um problema que existe. A escolha foi:

- menos peças para monitorar;
- deploy reproduzível com Docker e Kamal;
- banco e aplicação perto um do outro;
- recuperação simples numa VM substituta;
- **evoluir só quando uma métrica real pedir.**

Escolher a ferramenta grande antes do problema grande é a forma mais cara de errar.

O que isso custa, e é honesto admitir: uma VM é ponto único de falha. Volume Docker não é backup. O Postgres não é gerenciado — quem cuida dele é você.

2. Criar a VM na Azure

Cotas antes de tudo

A assinatura Azure for Students não oferece todos os tamanhos em todas as regiões. O portal responde:

NotAvailableForSubscription

O caminho certo:

1. **Assinaturas** → **Azure for Students** → **Uso + cotas**
2. filtre por `Compute`
3. veja as cotas regionais
4. escolha uma região onde o tamanho aparece

Não insista num tamanho marcado como indisponível. Pedir aumento de cota não resolve quando a oferta não está habilitada para a assinatura.

O assistente

Máquinas virtuais → **Criar** → **Máquina virtual do Azure**

Campo	Valor
Grupo de recursos	Novo: <code>rg-capacita-<seunome></code>
Nome	<code>vm-capacita-<seunome></code>
Região	uma com cota disponível
Imagem	Ubuntu Server 24.04 LTS — x64 Gen2
Tamanho	<code>Standard_B1s</code> (1 vCPU, 1 GiB) ou maior, se houver crédito
Autenticação	Chave pública SSH
Usuário	<code>azureuser</code>
Tipo de chave	Ed25519
Portas públicas	Nenhuma

"Nenhuma" é a escolha mais importante da tela. O assistente, se deixado, cria uma regra liberando SSH para a internet inteira. Bots varrem a porta 22 do IPv4 todo, o dia todo. Vamos abrir a 22 só para o seu IP.

Rede:

- VNet e sub-rede: aceite os padrões

- IP público: Standard
- NSG: avançado (é o firewall — vamos mexer nele)

Marque tudo com tags (`ambiente=capacitacao`, `dono=<seunome>`): é assim que você acha recurso órfão consumindo crédito depois.

Antes: duas chaves, um par

Criptografia assimétrica usa duas chaves que se completam. A **pública** você espalha; a **privada** nunca sai da sua máquina. O que uma fecha, só a outra abre.

Daí saem duas coisas diferentes:

- **sigilo** — eu fecho com a *sua* pública, e só você abre;
- **assinatura** — eu fecho com a *minha* privada, e todo mundo confere que fui eu.

É assim que o SSH funciona. Você põe a sua chave pública no servidor (`~/.ssh/authorized_keys`). Ao conectar, o servidor manda um desafio; você responde assinando com a privada; o servidor confere com a pública que já tinha. **A senha nunca trafega — nem existe.**

E é por isso que perder a chave privada é perder o acesso à máquina.

Compare com o JWT da Aula 3: lá a assinatura usa uma chave **simétrica** (HS256) — a mesma chave assina e confere, porque quem assina e quem confere são o mesmo servidor.

A chave privada

O portal oferece o download **uma vez**.

```
mkdir -p ~/.ssh
mv ~/Downloads/vm-capacita-seunome_key.pem ~/.ssh/azure-capacita
chmod 600 ~/.ssh/azure-capacita
```

`chmod 600` = só você lê. O SSH **recusa** usar uma chave com permissão frouxa.

Chave privada não vai por e-mail, não vai por WhatsApp, não vai para o Git. Nunca.

Abrir o SSH só para você

No NSG da VM, **Regras de segurança de entrada** → **Adicionar**:

Campo	Valor
Origem	Endereços IP
IPs de origem	SEU_IP/32
Portas de destino	22
Protocolo	TCP
Prioridade	100
Nome	allow-ssh-meu-ip

Descubra o seu IP:

```
curl -4 ifconfig.me
```

O `/32` significa "exatamente este endereço". Precisa também de:

Porta	Para quê
80	HTTP (o Cloudflare precisa alcançar)
443	HTTPS

Internet de casa troca de IP. Quando o SSH parar de conectar do nada, é isso: atualize a regra.

Primeiro acesso

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP_PUBLICO
```

3. Preparar o Ubuntu

Atualizar

```
sudo apt update && sudo apt upgrade -y
test -f /var/run/reboot-required && sudo reboot
```

Docker, do repositório oficial

O `apt install docker.io` do Ubuntu instala uma versão velha. Use o repositório da Docker:

```

sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

```

Libere o Docker para o `azureuser`:

```

sudo usermod -aG docker azureuser
exit

```

Saia e entre de novo — grupo só vale em sessão nova.

```

ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP
docker run --rm hello-world

```

Root e permissões

`root` é o usuário que pode tudo — sem "tem certeza?". Você trabalha como `azureuser` e chama `sudo` quando precisa.

Todo arquivo tem dono, grupo e três permissões: ler, escrever, executar.

```

chmod 600 arquivo    # dono lê e escreve; mais ninguém vê nada
chmod 700 pasta      # dono entra; mais ninguém

```

O SSH **exige** `600` numa chave privada — com permissão mais frouxa ele recusa usar o arquivo.

E nunca rode a aplicação como root: o Dockerfile já cria um usuário `rails` justamente para isso.

Swap

```

ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'sudo bash -s' < scripts/server-swap.sh

```

Numa VM de 1 GiB, Rails + Postgres + build estouram a RAM e o kernel mata o processo que estiver na frente (*OOM killer*), com uma mensagem que não explica nada. 2 GiB de swap resolvem.

Endurecer o SSH

Mantenha a sessão atual aberta enquanto testa em outro terminal.

```
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
EOF

sudo sshd -t          # valida a sintaxe ANTES de recarregar
sudo systemctl reload ssh
```

Em outro terminal:

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'echo OK'
```

Só feche a primeira sessão depois que isso responder. Errar a config do SSH com uma sessão só aberta é o jeito clássico de perder acesso à máquina.

4. Docker

Imagem × container

Imagem é a receita: o sistema, o Ruby, as gems, o seu código, congelados. **Container** é o bolo: uma instância rodando. Uma imagem, muitos containers.

O Dockerfile do Rails

O Rails 8 gera um Dockerfile de produção pronto. Vale ler:

```
FROM ruby:3.4.9-slim AS base
# ...

FROM base AS build
RUN apt-get install -y build-essential libpq-dev ...
COPY Gemfile Gemfile.lock ./
RUN bundle install
COPY . .

FROM base
RUN groupadd --system --gid 1000 rails && useradd rails --uid 1000 --gid 1000
USER 1000:1000
COPY --from=build "${BUNDLE_PATH}" "${BUNDLE_PATH}"
COPY --from=build /rails /rails
```

Duas decisões importantes:

Multi-stage. O estágio `build` instala compilador e headers para compilar as gems nativas. A imagem final copia só o resultado. Compilador não vai para produção: menos peso e menos ferramenta à mão de quem invadir.

Usuário não-root. Se alguém escapar da aplicação, cai num usuário sem privilégio. É uma linha que muda o tamanho do estrago.

`.dockerignore`

Diz o que **não** entra na imagem: `.git`, `log/`, `tmp/`, `node_modules`. Sem ele a imagem fica gorda e — pior — o histórico do Git vai junto.

Registry

O registry é o "GitHub das imagens". Usamos o **ghcr.io** (GitHub Container Registry): já vem com a conta e o Actions autentica sozinho com o `GITHUB_TOKEN`.

O fluxo: **Actions faz o build** → **empurra a imagem para o ghcr.io** → **a VM puxa e roda**.

A sua máquina nunca faz o build de produção.

5. Kamal

O Kamal (feito pela mesma turma do Rails) faz *zero-downtime deploy* com Docker, via SSH. Sem agente, sem painel: ele conecta na sua máquina e roda `docker`.

O que ele faz num `kamal deploy`:

1. builda a imagem e empurra para o registry
2. conecta na VM por SSH
3. puxa a imagem
4. sobe o container novo **ao lado** do antigo
5. espera o health check do novo passar
6. manda o tráfego para o novo e derruba o antigo

Falhou o health check? Ele não troca. O antigo continua servindo.

`config/deploy.yml`, por partes

```
service: automic-auth-api
image: <%= ENV.fetch("GHCR_USER") %>/automic-auth-api
```

O arquivo é ERB: dá para usar `ENV`. É por isso que este `deploy.yml` é **igual para todo mundo da turma** — o que muda vem das variables do Actions.

```
ssh:
  user: azureuser
run_directory: /home/azureuser/.kamal
```

Sem login root, então o Kamal precisa de um lugar para gravar lock e auditoria.

```
env:
  clear:
    SOLID_QUEUE_IN_PUMA: "true"
    WEB_CONCURRENCY: "1"
    DB_HOST: automic-auth-api-db
  secret:
    - RAILS_MASTER_KEY
    - JWT_SECRET
```

- `clear` vai como texto no comando `docker run`. Configuração, não segredo.
- `secret` vai por um arquivo de ambiente, com permissão restrita no servidor.

Colocar `JWT_SECRET` no `clear` seria o mesmo que publicá-lo: ele apareceria em `docker inspect` e no histórico do shell.

`DB_HOST: automic-auth-api-db` é o **nome do container** do banco. Containers na mesma rede Docker se enxergam por nome — não precisa de IP.

```
accessories:
  db:
    image: postgres:16
    directories:
      - data:/var/lib/postgresql/data
```

Accessory é um container que o Kamal gerencia mas não faz deploy junto com o app. O `directories` cria o volume: é ele que faz os dados sobreviverem a deploy e a rebuild.

`kamal deploy` **não** sobe accessories. Depois de mudar env de accessory: `kamal accessory reboot db`.

As três camadas de segredo

```
GitHub Actions Secrets
  ↓ (viram variáveis de ambiente no runner)
.kamal/secrets
  ↓ (referencia as variáveis – nenhum valor aqui, por isso vai para o Git)
config/deploy.yml (env.secret)
  ↓ (o Kamal injeta no container)
ENV["JWT_SECRET"] no Rails
```

Nenhum valor real toca o repositório em nenhum ponto.

6. Segredos do Rails

`credentials` e `master.key`

```
bin/rails credentials:edit
```

O Rails abre um YAML no editor, e ao salvar grava `config/credentials.yml.enc` — criptografado, seguro no Git. A chave que decripta é `config/master.key`, que **nunca** vai para o Git (já está no `.gitignore`).

Em produção, `master.key` vira o secret `RAILS_MASTER_KEY`.

Antes: o que é uma variável de ambiente

Um par `nome=valor` que o sistema operacional entrega ao processo quando ele sobe.

```
export JWT_SECRET=abc123
```

E o programa lê com `ENV["JWT_SECRET"]`.

Por que não deixar o valor no código? Porque o código vai para o Git. A ideia é: mesmo código, ambientes diferentes, valores diferentes. É um dos [12 fatores](#), e é como o Kamal injeta segredo no container.

Credentials × ENV

	Quando
Credentials	segredo estável, que é do app: chave de API de terceiro
ENV	o que muda por ambiente ou por máquina: senha do banco, host de SMTP

Este projeto usa ENV para quase tudo, porque quem provisiona (Kamal) já sabe injetar.

Gere o `JWT_SECRET`:

```
bin/rails secret
```

7. GitHub Actions

CI: o portão

`.github/workflows/ci.yml` roda em todo push e PR:

- `scan_ruby` — Brakeman (segurança estática) e bundler-audit (CVE em gems)

- `lint` — RuboCop
- `test` — `bin/rails test` contra um Postgres descartável

Se qualquer um falhar, o código não entra na `main`. Isso é o que faz "deploy automático" não ser "publicar bug automático".

Deploy: o interessante

O ponto delicado é o SSH. A porta 22 está fechada para a internet. O runner do GitHub tem IP diferente a cada execução — não dá para liberar antes.

A sequência:

1. autentica na Azure (OIDC)
2. descobre o próprio IP público
3. cria uma regra no NSG liberando a 22 só para esse IP/32
4. faz o deploy
5. APAGA a regra — mesmo se o deploy falhar

O passo 5 tem `if: always()`. **Deixar a 22 aberta é o erro que transforma um deploy ruim num incidente de segurança.**

8. OIDC — por que não usar senha

O jeito comum seria criar um *client secret* na Azure e guardar como secret do GitHub. Problemas: ele é longo, vale por meses, e quem tiver acesso ao repositório tem acesso à sua Azure.

OIDC (OpenID Connect) inverte: o GitHub emite, a cada execução, um token de curta duração que diz "sou o workflow do repositório X, na branch Y". A Azure verifica e devolve um acesso temporário.

Não existe segredo de longa duração em lugar nenhum.

Criando

1. Registro de aplicativo — Microsoft Entra ID → Registros de aplicativo → Novo registro:

- nome: `github-capacita-<seunome>`
- contas: somente este diretório
- URI de redirecionamento: vazia
- **não crie segredo do cliente**

Anote: Application (client) ID, Directory (tenant) ID, Subscription ID.

2. Credencial federada — no app criado, Certificados e segredos → Credenciais federadas → Adicionar → cenário **GitHub Actions**:

Campo	Valor
Organização	seu usuário do GitHub
Repositório	nome do repositório
Tipo de entidade	Branch
Branch	<code>main</code>

O subject resultante:

```
repo:SEU-USUARIO/SEU-REPO:ref:refs/heads/main
```

Precisa bater **caractere por caractere** com o que o GitHub emite. Erro de maiúscula, de nome ou de branch dá `AADSTS700213` — e a mensagem não ajuda.

3. Permissão mínima — no **NSG** (não na assinatura), IAM → Adicionar atribuição de função → **Colaborador de Rede** → o app criado.

Escopo no NSG e não na assinatura: se a credencial vazar, o estrago se limita a regras de firewall daquela máquina.

4. Chave SSH de deploy — separada da sua chave pessoal:

```
ssh-keygen -t ed25519 -f ~/.ssh/capacita-github -C "github-actions" -N ""

ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP \
'umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys' < ~/.ssh/capacita-github.pub
```

Chave separada dá para revogar o acesso do CI sem trocar o seu.

Cadastrando no GitHub

Settings → **Secrets and variables** → **Actions**

Secrets:

Nome	Conteúdo
<code>AZURE_CLIENT_ID</code>	Application (client) ID
<code>AZURE_TENANT_ID</code>	Directory (tenant) ID
<code>AZURE_SUBSCRIPTION_ID</code>	Subscription ID
<code>AZURE_SSH_PRIVATE_KEY</code>	conteúdo de <code>~/.ssh/capacita-github</code> (a privada)
<code>RAILS_MASTER_KEY</code>	conteúdo de <code>config/master.key</code>
<code>AUTOMIC_AUTH_API_DATABASE_PASSWORD</code>	invente uma senha longa
<code>JWT_SECRET</code>	saída de <code>bin/rails secret</code>
<code>SMTP_ADDRESS</code> / <code>SMTP_USERNAME</code> / <code>SMTP_PASSWORD</code>	do provedor de e-mail
<code>KAMAL_PROXY_SSL_CERTIFICATE</code>	certificado Origin CA
<code>KAMAL_PROXY_SSL_PRIVATE_KEY</code>	chave do Origin CA

Variables (não são segredo):

Nome	Exemplo
<code>GHC_USER</code>	seu usuário do GitHub
<code>AZURE_VM_IP</code>	<code>57.156.65.151</code>
<code>APP_HOST</code>	<code>seunome.capacita.exemplo.tech</code>
<code>AZURE_RESOURCE_GROUP</code>	<code>rg-capacita-seunome</code>
<code>AZURE_NSG_NAME</code>	nome do NSG
<code>CORS_ORIGINS</code>	<code>http://localhost:5173</code>
<code>MAILER_FROM</code>	<code>Capacitação <noreply@exemplo.tech></code>

Pelo CLI (não deixa o valor no histórico do shell):

```
gh secret set JWT_SECRET
gh secret set KAMAL_PROXY_SSL_CERTIFICATE < origin-ca.pem
gh variable set APP_HOST
```

9. Cloudflare e TLS

HTTPS é HTTP dentro de um túnel

O HTTP da Aula 1 é **texto puro**: quem estiver no caminho — o roteador do café, o provedor — lê tudo, inclusive a senha. O **TLS** embrulha esse texto num túnel criptografado.

Mesmo protocolo, mesmos verbos, mesmos cabeçalhos, só que fechado. O "S" de HTTPS é isso, e nada além disso.

O handshake, em três passos

1. O servidor apresenta o **certificado** dele.
2. O cliente confere se confia em **quem assinou** aquele certificado.
3. Os dois combinam uma chave temporária e conversam com ela.

O par de chaves assimétricas só serve para o passo 3. Depois disso a conversa usa criptografia **simétrica**, que é muito mais rápida.

Certificado e autoridade

Um certificado diz: *"esta chave pública pertence a este domínio"* — e vem **assinado por uma Autoridade Certificadora (CA)**.

O seu navegador já nasce com uma lista de CAs em que confia. Confia na CA → confia em quem ela assinou. É uma cadeia.

- **Autoassinado** é você jurando que é você. O navegador não aceita.
- **Let's Encrypt** é uma CA pública e gratuita, em que os navegadores confiam.
- **Cloudflare Origin CA** *não* é pública — e é exatamente por isso que ela funciona aqui, como você vai ver.

Proxy reverso

Um proxy comum fica na frente do **cliente**. Um proxy **reverso** fica na frente do **servidor**: recebe tudo na 443, termina o TLS, e repassa para a aplicação em HTTP interno.

Assim o Rails não precisa saber nada de certificado. Na nossa arquitetura há dois:

```
navegador → Cloudflare → kamal-proxy (na sua VM) → Rails
              (proxy 1)      (proxy 2)
```

É por isso que `config.assume_ssl = true`: ele avisa o Rails de que o "http" que chegou já veio de um "https" lá fora, e o Rails para de montar URLs erradas.

DNS

No Cloudflare, no domínio da capacitação:

Campo	Valor
Tipo	A
Nome	seunome.capacita
Conteúdo	o IP público da sua VM
Proxy	Proxied (nuvem laranja)

Proxied significa que o tráfego passa pelo Cloudflare antes de chegar na sua VM. Você ganha cache, proteção contra DDoS e — o principal aqui — o IP real da sua máquina não aparece no DNS.

Certificado Origin CA

SSL/TLS → **Origin Server** → Create Certificate. O Cloudflare gera o par e mostra **uma vez**. Copie os dois para os secrets `KAMAL_PROXY_SSL_CERTIFICATE` e `KAMAL_PROXY_SSL_PRIVATE_KEY`.

Full (strict)

SSL/TLS → Overview → **Full (strict)**.

Os três modos:

Modo	Cloudflare → sua VM
Flexible	em texto puro — não use
Full	criptografado, mas aceita certificado inválido
Full (strict)	criptografado e o certificado é validado

Com `Flexible`, o cadeado aparece para o usuário e a senha dele trafega em claro no último trecho. É pior que não ter HTTPS, porque mente.

Por que não Let's Encrypt

O Kamal sabe emitir Let's Encrypt sozinho. Aqui não dá:

- com o DNS **proxied**, o desafio HTTP-01 não chega ao seu servidor como o Let's Encrypt espera;
- daria para tirar o proxy durante a emissão, mas isso expõe o IP e vira ritual manual a cada renovação.

O Origin CA resolve: protege o trecho Cloudflare → Azure, dispensa desafio, funciona com Full (strict) e vale anos. O certificado que o **usuário** vê é o público do Cloudflare — o Origin CA nunca aparece para o navegador (e por isso não precisa ser confiável por ele).

10. O primeiro deploy

```
GHCR_USER=seu-usuario AZURE_VM_IP=1.2.3.4 APP_HOST=seunome.capacita.exemplo.tech \
bundle exec kamal config
```

Isso valida o `deploy.yml` sem tocar em nada. Depois:

Actions → Deploy (Kamal) → Run workflow → command: `setup`

`setup` instala o Docker se faltar, sobe os accessories e faz o primeiro deploy. Só na primeira vez — depois é `deploy`.

Deu certo:

```
curl https://seunome.capacita.exemplo.tech/api/v1/status
```

```
{"status":"ok","service":"atomic-auth-api","environment":"production"}
```

A partir daqui, `git push` na `main` faz deploy sozinho.

11. Operar

```
export GHCR_USER=... AZURE_VM_IP=... APP_HOST=...

kamal app logs -f           # logs ao vivo
kamal app exec --interactive --reuse "bin/rails console"
kamal app exec "bin/rails db:migrate"
kamal app details           # o que está rodando
kamal accessory reboot db   # depois de mudar env do banco
kamal rollback              # volta para a versão anterior
```

Na VM:

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP
docker ps
free -h
df -h
```

Backup

Volume Docker não é backup. Se a VM sumir, o volume some junto.

```
ssh azureuser@SEU_IP \
'docker exec atomic-auth-api-db pg_dump -U atomic_auth_api atomic_auth_api_production' \
| gzip > backup-$(date +%F).sql.gz
```

E, o que quase ninguém faz: **restaure num banco de teste**. Backup que nunca foi restaurado não é backup, é esperança.

Exercício da aula

1. Crie a VM na Azure, sem porta pública nenhuma.
 2. Abra a 22 só para o seu IP, mais a 80 e a 443.
 3. Instale o Docker e rode o `hello-world`.
 4. Rode o `server-swap.sh` e endureça o SSH.
 5. Configure o OIDC e cadastre todos os secrets e variables.
 6. Crie o registro DNS e o certificado Origin CA. Ligue o Full (strict).
 7. Rode o workflow com `command: setup`.
 8. `curl https://seunome.capacita.<domínio>/api/v1/status`
 9. Faça o fluxo completo da Aula 3 **contra a sua API em produção**: cadastro, e-mail de verdade na sua caixa de entrada, login, logout, recuperação de senha.
 10. Mude a mensagem da rota de status, dê push na `main` e veja o deploy acontecer sozinho.
-

Recapitulando

- VPS: você controla tudo, e opera tudo.
 - Escolha a ferramenta do tamanho do problema. Kubernetes não era o tamanho.
 - Imagem é receita, container é bolo. Multi-stage e usuário não-root.
 - `clear` é configuração, `secret` é segredo — e a diferença aparece no `docker inspect`.
 - Volume é o que faz o dado sobreviver ao deploy. E ainda assim não é backup.
 - OIDC troca segredo de longa duração por token de curta. Prefira sempre.
 - A porta 22 abre por um minuto e fecha — inclusive quando dá errado.
 - Full (strict), sempre. `Flexible` mente para o usuário.
-

O que o `seem-backend` tem a mais

O que ficou de fora daqui, e por quê:

Recurso	Para quê
Datadog (logs)	logs centralizados e Error Tracking, com o app logando JSON estruturado
rack-attack	bloqueia scanner e rajada de erro por IP — a internet bate na sua porta o dia todo
Active Storage	foto de perfil, em volume Docker persistente
Solid Queue dedicado	processamento em background
Audit log	histórico de toda mutação feita por admin
Export .xlsx	relatório de presença
OpenAPI/Swagger	contrato da API documentado

Nenhum deles é difícil depois do que você viu aqui. Todos estão no [seem-backend](#) — agora dá para ler.